Graph Theory

Mid-Term Report

Submitted by

Teenu Anand

Summer Of Science

June 2026

Contents

1 Introduction **4**
- 1.1 Graph . . . 4
- 1.2 Loop . . . 4
- 1.3 Degree of vertex . . . 4
- 1.4 adjacency . . . 4

2 Types of Graphs **5**
- 2.1 Simple graph . . . 5
- 2.2 Directed graph . . . 5
- 2.3 Some common graphs . . . 5

3 Graph representations **6**
- 3.1 Adjacency Matrices . . . 6
- 3.2 Adjacency lists . . . 6
- 3.3 Representation in C++ . . . 6

4 Graph traversal methods and recursive graph exploration **7**
- 4.1 Breadth First Search(BFS) . . . 7
  - 4.1.1 How BFS Works . . . 7
  - 4.1.2 BFS C++ code . . . 7
- 4.2 Depth First Search(DFS) . . . 8
  - 4.2.1 How DFS Works . . . 8
  - 4.2.2 Recursive DFS code . . . 8

5 Connected Components **8**
- 5.1 Why BFS/DFS is Used . . . 8

6 Graph Traversal Trees **9**
- 6.1 BFS Traversal Tree . . . 9
- 6.2 DFS Traversal Tree . . . 9
- 6.3 Applications of Traversal Trees . . . 9

7 Bipartite Graphs **10**
- 7.1 Theorem . . . 10
  - 7.1.1 Proof . . . 10

8 Matching Problems **11**
- 8.1 Sex in America: An Application of Bipartite Graphs . . . 11
  - 8.1.1 Modeling the Problem . . . 11
  - 8.1.2 Degree of a Vertex . . . 11
  - 8.1.3 Key Observation . . . 11
  - 8.1.4 Average Number of Partners . . . 11
  - 8.1.5 Interpretation . . . 12

9 Hand Shaking Lemma **12**
- 9.1 Proof . . . 12
- 9.2 Bipartite Matchings . . . 12
  - 9.2.1 Matching in Bipartite Graphs . . . 12

9.2.2 Hall's Matching Theorem . . . 13
9.3 Hall's Matching Theorem . . . 13
9.3.1 Bottlenecks . . . 13
9.3.2 Degree-Constrained Bipartite Graphs . . . 13
9.3.3 Regular Bipartite Graphs . . . 14
9.3.4 Applications . . . 14

10 Graph Coloring and Chromatic Number 14
10.1 Graph Coloring . . . 14
10.2 Chromatic Number . . . 15
10.3 Examples . . . 15
10.3.1 Complete Graph . . . 15
10.3.2 Cycle Graphs . . . 15
10.3.3 Bipartite Graphs . . . 15
10.3.4 Trees . . . 16
10.4 Important Results . . . 16
10.5 Greedy Coloring Algorithm . . . 16
10.6 Applications of Graph Coloring . . . 16

11 Paths 17
11.1 Length of a Path . . . 17
11.2 Types of paths . . . 17
11.2.1 Simple Path . . . 17
11.2.2 Closed Path . . . 17
11.2.3 Directed Path . . . 17

12 Walks 17
12.1 Length of a walk . . . 17

13 Shortest Paths 17
13.1 Algorithms for Finding Shortest Paths . . . 18
13.1.1 Breadth-First Search (BFS) . . . 18
13.1.2 Dijkstra's Algorithm . . . 18

14 Connectivity in Graphs 18
14.1 Connected Graph . . . 18
14.2 Disconnected graph . . . 18
14.3 connected Components . . . 18
14.4 Strong and Weak Connectivity (Directed Graphs) . . . 18

15 Relation Between Adjacency Matrices and Walks in Graphs 18

16 Implementing Shortest Path Algorithm for Unweighted Graphs 19
16.1 Breadth-First Search (BFS) . . . 19
16.1.1 pseudo code . . . 19

17 Weighted Graph 20
17.1 Dijkatra's algorithm . . . 20
17.1.1 Pseudo code . . . 20

18 Trees **21**
18.1 Properties of Trees 21
18.2 Terminology 21
18.3 Types of trees 21

19 Spanning trees **22**

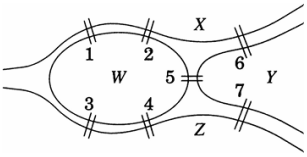
20 Properties **22**

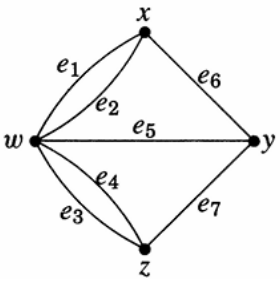
21 Minimum spanning trees **22**
21.1 Properties 23
21.2 Disjoint set union 23
21.2.1 Operations in DSU 23
21.2.2 DSU representation 23
21.2.3 Optimisations 24
21.2.4 Algorithms 24
21.2.5 Applications of DSU 25
21.3 Kruskal's algorithm 25

22 Applications of Spanning Trees in Optimization and Networks **25**

1 Introduction

The birth of graph theory will suggest our basic definition of graph.
1.1.1. Example. *The Königsberg Bridge Problem.* The city of Königsberg was located on the Pregel river in Prussia. The city occupied two islands plus areas on both banks. These regions were linked by seven bridges as shown on the left below. The citizens wondered whether they could leave home, cross every bridge exactly once, and return home. The problem reduces to traversing the figure on the right, with heavy dots representing land masses and curves representing bridges.
The model on the right makes it easy to argue that the desired traversal does not exist.





Each time we enter and leave a land mass, we use two bridges ending at it. We can also pair the first bridge with the last bridge on the land mass where we begin and end. Thus existence of the desired traversal requires that each land mass be involved in an even number of bridges. This necessary condition did not hold in Königsberg. ■

1.1 Graph

A graph G is a triple consisting of vertex set V(G), an edge set E(G),and a relation that associates with each edge two vertices(not necessarily distinct) called its endpoints.

1.2 Loop

A loop is an edge whose endpoints are equal.
Multiple edges are edges having the same pair of endpoints.

1.3 Degree of vertex

, and an edge is said to be incident to the vertices it joins. The number of edges incident to a vertex v is called the degree of the vertex and is denoted by deg(v); equivalently, the degree of a vertex is equals the number of vertices adjacent to it.

1.4 adjacency

Two vertices in a simple graph are said to be adjacent if they are joined by an edge

2 Types of Graphs

2.1 Simple graph

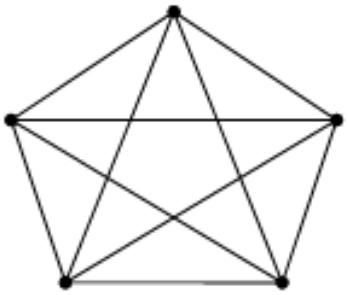
A simple graph is a graph having no loops or multiple edges
When u and v are endpoints of an edge,they are adjacent and are neighbors.

2.2 Directed graph

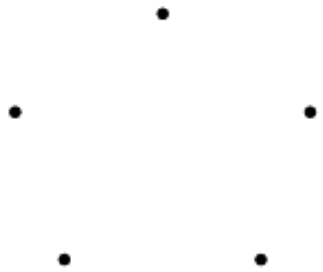
A directed graph (or digraph)(V,E) consists of a nonempty set of vertices V and a set of directed edges (or arcs) E. Each directed edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair (u,v) is said to start at u and end at v.

2.3 Some common graphs

Some graphs come up so frequently that they have names. The complete graph on n vertices, denoted Kn, has an edge between every two vertices, for a total of $n \cdot n - 1/2 = edges$
The empty graph has no edges at all. The empty graph has no edges at all.



(a) Complete graph (K_5)



(b) Empty graph with 5 nodes

The n-node graph containing n-1 edges in sequence is known has the line graph L_n

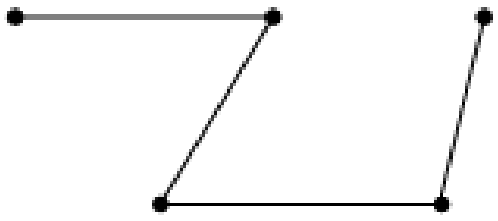


Figure 3: A 5-node line graph L_5

3 Graph representations

3.1 Adjacency Matrices

There are many ways to represent a graph.you can draw it or you can represent it with sets.another way is to represent it with adjacency matrix
Given an n-node graph $G = (V, E)$ where $V = \{v_1, v_2, ..., v_n\}$,the adjacency matrix for G is the $n \times n$ matrix $A_G = \{a_{ij}\}$ where

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise} \end{cases}$$

3.2 Adjacency lists

Specifies the vertices adjacent to each vertex of graph.

3.3 Representation in C++

Graph implementation in C++ using an adjacency list.

```
#include <iostream>
#include <vector>
#include <list>

using namespace std;

class Graph {
    int V;
    vector<vector<pair<int, int>>> adj;

public:
    Graph(int vertices) {
        V = vertices;
        adj.resize(V);
    }

    void addEdge(int u, int v, int weight = 1) {
        adj[u].push_back({v, weight});
        adj[v].push_back({u, weight});
    }
    void printGraph() {
        for (int i = 0; i < V; i++) {
            cout << "Vertex " << i << " -> ";

            for (auto edge : adj[i]) {
                cout << "(" << edge.first
                     << ", weight = "
                     << edge.second << ") ";
```

```
            }

            cout << endl;
        }
    }
};
```

4 Graph traversal methods and recursive graph exploration

4.1 Breadth First Search(BFS)

BFS is a graph traversal algorithm that visits nodes level by level starting from a source node.
It explores all immediate neighbors first, then neighbors of those neighbors, and so on.

4.1.1 How BFS Works

1. Put the starting node into queue
2. Mark it as visited.
3. Repeatedly:
    - Remove a node from the queue.
    - Visit all its unvisited neighbors. Add those neighbors to the queue.

4.1.2 BFS C++ code

```
void bfs(int start, vector<vector<int>>& adj) {
    vector<bool> visited(adj.size(), false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        cout << node << " ";

        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
```

```
    }
}
```

4.2 Depth First Search(DFS)

DFS is a graph traversal algorithm that explores a path as deeply as possible before backtracking.
Instead of visiting nodes level by level like BFS, DFS keeps moving to an unvisited neighbor until it can't go further.

4.2.1 How DFS Works

1. Visit the current node.
2. Mark it as visited.
3. Go to an unvisited neighbor.
4. Repeat until no unvisited neighbors remain.
5. Backtrack to the previous node and continue.

4.2.2 Recursive DFS code

```
    void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true;
    cout << node << " ";

    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, adj, visited);
        }
    }
}
```

5 Connected Components

A connected component in an undirected graph is a group of vertices where every vertex can be reached from every other vertex in that group.

5.1 Why BFS/DFS is Used

1. Run DFS/BFS from current node.
2. Mark all reachable node.
3. Count one connected component
4. Repeat for remaining unvisited nodes

```
int countComponents(int n,
                    vector<vector<int>>& adj)
{
    vector<bool> visited(n, false);
    int count = 0;

    for(int i = 0; i < n; i++)
    {
        if(!visited[i])
        {
            dfs(i, adj, visited);
            count++;
        }
    }

    return count;
}
```

6 Graph Traversal Trees

A graph traversal tree is the tree formed by the edges used during a BFS or DFS traversal.

6.1 BFS Traversal Tree

A BFS traversal tree is constructed when Breadth-First Search is performed on a graph. Starting from a source vertex, BFS explores all neighboring vertices before moving to the next level of vertices. The edges used to reach previously unvisited vertices form the BFS traversal tree. This tree represents the shortest-path structure from the source vertex in an unweighted graph.

6.2 DFS Traversal Tree

A DFS traversal tree is constructed when Depth-First Search is performed on a graph. DFS explores as far as possible along each branch before backtracking. The edges used to discover new vertices form the DFS traversal tree. Unlike a BFS tree, a DFS tree does not necessarily represent shortest paths but illustrates the recursive exploration of the graph

6.3 Applications of Traversal Trees

- Finding connected components in a graph.
- Detecting cycles.
- Finding paths between vertices.
- Topological sorting of directed acyclic graphs
- Network analysis and routing.

- Constructing spanning trees.

7 Bipartite Graphs

A bipartite graph is a graph where the vertices can be split into two groups such that:

- Every edge connects a vertex from one group to a vertex in the other group.
- No edge exists between two vertices in the same group.

Equivalently, a graph is bipartite if it can be colored using two colors so that no two adjacent vertices share the same color.

7.1 Theorem

A graph is bipartite if and only if it contains no odd-length cycle.

7.1.1 Proof

We has to prove both directions.
1. If a graph is bipartite, then it has no odd cycle
Assume the graph is bipartite.
Let the two sets be:U and V.
Now suppose there is a cycle.as the graph is bipartite vertices on the cycle alternate between U and V.to return to initial vertex the number of edges traveled must be even.
Therefore every cycle in a bipartite graph has even length.
Hence odd cycles cannot exist.
2. If a graph has no odd cycle,then it is bipartite
Choose any vertex r.
Define:

- set U:vertices at even distance from r.
- set V:vertices at odd distance from r.

We claim every edge joins a vertex in U to one in V.
Suppose not.Assume there is an edge connecting two vertices in the same set.
Case 1:Both vertices are in U As both there distances fron r is even.r and those 2 vertices form a odd length cycle.
Hence Contradiction!
Case 2:Both vertices are in V Similarly,odd+odd+1 is also odd producing odd cycle.
Contradiction.
Therefore no edge can connect two vertices in the same set.
Hence every edge goes between U and V.

So the graph is bipartite.

8 Matching Problems

8.1 Sex in America: An Application of Bipartite Graphs

Graph theory can be applied to analyze real-world social networks. One interesting example is the study of opposite-gender sexual partnerships in the United States.

8.1.1 Modeling the Problem

Let $G = (V, E)$ be a graph where each vertex represents a person in America. The vertex set is divided into two disjoint sets:

- M = set of all males
- F = set of all females

An edge is drawn between a male and a female if they have been sexual partners. Since edges only connect vertices from different sets, G is a **bipartite graph**.

8.1.2 Degree of a Vertex

The degree of a vertex represents the number of opposite-gender partners of that person.

- Degree of a male vertex = number of female partners.
- Degree of a female vertex = number of male partners.

8.1.3 Key Observation

Every partnership corresponds to exactly one edge in the graph. Each edge is incident to one male vertex and one female vertex. Therefore, the sum of the degrees of all male vertices equals the sum of the degrees of all female vertices:

$$\sum_{x \in M} \deg(x) = \sum_{y \in F} \deg(y)$$

8.1.4 Average Number of Partners

The average number of opposite-gender partners for males is

$$\frac{\sum_{x \in M} \deg(x)}{|M|}$$

and the average number of opposite-gender partners for females is

$$\frac{\sum_{y \in F} \deg(y)}{|F|}.$$

Dividing the degree-sum equation by $|M||F|$, we obtain

$$\frac{\sum_{x \in M} \deg(x)}{|M|} \frac{|F|}{|M|} \cdot \frac{\sum_{y \in F} \deg(y)}{|F|}.$$

Thus,

$$\text{Average Male Partners} \frac{|F|}{|M|} \times \text{Average Female Partners}.$$

8.1.5 Interpretation

According to U.S. Census data, there are slightly more females than males. Hence,

$$\frac{|F|}{|M|} \approx 1.035.$$

Therefore, the average number of opposite-gender partners for males can only be about 3.5 % higher than that for females.

9 Hand Shaking Lemma

The sum of the degrees of the vertices in a graph equals twice the number of edges

9.1 Proof

Every edge contributes two to the sum of the degrees, one for each of its endpoints.

9.2 Bipartite Matchings

A bipartite graph is a graph whose vertices can be partitioned into two disjoint sets L and R such that every edge connects a vertex in L to a vertex in R.

A **bipartite graph** is a graph $G = (V, E)$ together with a partition of its vertex set into two subsets L and R such that every edge has one endpoint in L and the other endpoint in R.

Bipartite graphs are widely used to model relationships between two different types of objects, such as students and courses, workers and jobs, or men and women.

9.2.1 Matching in Bipartite Graphs

A matching is a set of edges that do not share any common vertex. In a bipartite graph, a matching represents a valid assignment between the two sets of vertices.

A **matching** in a graph is a set of edges such that no two edges have a common endpoint.

For example, in a graph representing men and women, a matching corresponds to pairing each man with a distinct woman.

A matching is said to **cover** a set of vertices L if every vertex in L is incident to an edge in the matching.

A **perfect matching** is a matching that covers every vertex of the graph.

9.2.2 Hall's Matching Theorem

A natural question is whether it is possible to find a matching that covers all vertices on one side of a bipartite graph.

To answer this question, Hall's Matching Theorem provides a necessary and sufficient condition.

For a subset $S \subseteq L$, the neighborhood of S, denoted by $N(S)$, is the set of all vertices in R adjacent to at least one vertex in S:

$$N(S) = r \in R \mid (s, r) \in E \text{ for some } s \in S.$$

9.3 Hall's Matching Theorem

Let G be a bipartite graph with vertex partition L and R. There exists a matching that covers L if and only if

$$|N(S)| \geq |S|$$

for every subset $S \subseteq L$.

This condition means that every group of vertices in L collectively has at least as many available neighbors in R as its size.

9.3.1 Bottlenecks

A subset $S \subseteq L$ is called a **bottleneck** if

$$|S| > |N(S)|.$$

If a bottleneck exists, then Hall's condition fails and a matching covering L cannot exist.

9.3.2 Degree-Constrained Bipartite Graphs

Checking Hall's condition for every subset can be difficult. A simpler sufficient condition is based on vertex degrees.

A bipartite graph with partition L, R is called **degree-constrained** if

$$\deg(l) \geq \deg(r)$$

for every $l \in L$ and $r \in R$.

If a bipartite graph G with partition L, R satisfies $|L| \leq |R|$ and is degree-constrained, then there exists a matching that covers L.

This theorem guarantees a matching without explicitly checking Hall's condition.

9.3.3 Regular Bipartite Graphs

A graph is called **regular** if every vertex has the same degree.

Examples include 2-regular graphs and 3-regular graphs, where every vertex has degree 2 or 3 respectively.

Every regular bipartite graph has a perfect matching.

Since all vertices have equal degree, the graph automatically satisfies the degree-constrained property. Therefore, Hall's theorem ensures the existence of a matching that covers all vertices, resulting in a perfect matching.

9.3.4 Applications

Bipartite matching has many practical applications:

- Job assignment problems
- Student-course allocation
- Resource allocation
- Network design
- Marriage and pairing problems
- Scheduling and task assignment

Thus, bipartite matchings provide an important connection between graph theory and real-world optimization problems.

10 Graph Coloring and Chromatic Number

Graph coloring is one of the most important topics in graph theory. It involves assigning colors to the vertices of a graph subject to certain constraints. Graph coloring has numerous applications in scheduling, resource allocation, map coloring, and network design.

10.1 Graph Coloring

Definition 1 *A **vertex coloring** of a graph $G = (V, E)$ is an assignment of colors to the vertices such that no two adjacent vertices receive the same color.*

A coloring satisfying this condition is called a **proper coloring**.

For example, if two vertices are connected by an edge, they must be assigned different colors.

10.2 Chromatic Number

Definition 2 *The **chromatic number** of a graph G, denoted by $\chi(G)$, is the minimum number of colors required to properly color the vertices of the graph.*

The chromatic number measures how difficult it is to color a graph while satisfying the coloring constraints.

10.3 Examples

10.3.1 Complete Graph

A complete graph K_n contains an edge between every pair of vertices.

Since every vertex is adjacent to every other vertex, each vertex must receive a different color.

Therefore,

$$\chi(K_n) = n.$$

For example,

$$\chi(K_4) = 4.$$

10.3.2 Cycle Graphs

The chromatic number of a cycle depends on its length.

- If the cycle has even length, then

$$\chi(C_n) = 2.$$

“

- If the cycle has odd length, then

$$\chi(C_n) = 3.$$

“

For example,

$$\chi(C_4) = 2, \qquad \chi(C_5) = 3.$$

10.3.3 Bipartite Graphs

A bipartite graph can be divided into two disjoint vertex sets such that every edge joins vertices from different sets.

Therefore, every bipartite graph can be colored using two colors.

Hence,

$$\chi(G) = 2$$

for every connected bipartite graph containing at least one edge.

10.3.4 Trees

A tree is an acyclic connected graph.

Every tree is bipartite, and therefore can be colored using two colors.

Thus,

$$\chi(T) = 2$$

for every tree with at least two vertices.

10.4 Important Results

Theorem 1 *A graph is bipartite if and only if it is 2-colorable.*

This theorem establishes a direct relationship between bipartite graphs and graph coloring.

Theorem 2 *A graph is bipartite if and only if it contains no odd cycle.*

Since odd cycles require three colors, the absence of odd cycles guarantees that two colors are sufficient.

10.5 Greedy Coloring Algorithm

One simple method for coloring a graph is the greedy coloring algorithm.

1. Consider the vertices one by one.
2. Assign the smallest available color that has not been used by any adjacent vertex.
3. Continue until all vertices are colored.

Although the greedy algorithm does not always produce an optimal coloring, it often provides a good approximation.

10.6 Applications of Graph Coloring

Graph coloring has many practical applications:

- Timetable scheduling
- Exam scheduling
- Register allocation in compilers
- Frequency assignment in wireless networks
- Map coloring
- Resource allocation problems

11 Paths

A path in a graph is a sequence of vertices connected by edges. It represents a route from one vertex to another without necessarily repeating vertices.
A path is an ordered sequence of vertices $v_1, v_2, ..., v_n$ such that every pair of consecutive vertices is connected by an edge.

11.1 Length of a Path

The length of a path is the number of edges it contains.

11.2 Types of paths

11.2.1 Simple Path

A path in which no vertex is repeated.

11.2.2 Closed Path

A path that starts and ends at the same vertex.

11.2.3 Directed Path

In a directed graph, all edges are followed in their specified direction.

12 Walks

A walk is a sequence of vertices and edges in which each edge connects the preceding vertex to the following vertex. Unlike a path, a walk may repeat vertices and edges.
A walk is an ordered sequence of vertices $v_1, v_2, ..., v_n$such that every pair of consecutive vertices is connected by an edge. Vertices and edges can be visited multiple times.

12.1 Length of a walk

The length of a walk is the number of edges traversed.

13 Shortest Paths

A shortest path is the path between two vertices that has the minimum total distance or cost among all possible paths connecting them.

1. In an unweighted graph, the shortest path is the one with the fewest edges.
2. In a weighted graph, the shortest path is the one with the minimum sum of edge weights.

13.1 Algorithms for Finding Shortest Paths

13.1.1 Breadth-First Search (BFS)

Used for unweighted graphs.

13.1.2 Dijkstra's Algorithm

Finds the shortest path in weighted graphs with non-negative edge weights.

14 Connectivity in Graphs

Connectivity refers to the ability to reach one vertex from another through a sequence of edges in a graph. It indicates how well the vertices in a graph are linked

14.1 Connected Graph

A graph is called connected if there is at least one path between every pair of vertices. In other words, every vertex can be reached from any other vertex.

14.2 Disconnected graph

A graph is called disconnected if at least one pair of vertices has no path connecting them. Such a graph consists of two or more separate components.

14.3 connected Components

A connected component is a maximal connected subgraph in which every pair of vertices is connected by a path.

14.4 Strong and Weak Connectivity (Directed Graphs)

For directed graphs, connectivity is classified into two types:

1. Strongly Connected Graph: A directed graph is strongly connected if there is a directed path from every vertex to every other vertex.

2. Weakly Connected Graph: A directed graph is weakly connected if replacing all directed edges with undirected edges results in a connected graph.

15 Relation Between Adjacency Matrices and Walks in Graphs

The adjacency matrix of a graph provides an efficient way to determine the number of walks between vertices.
An adjacency matrix A is a square matrix where:

1. $A_{ij} = 1$ if there is an edge between vertices i and j.

2. $A_{ij} = 0$ if there is no edge.

The powers of the adjacency matrix reveal information about walks in the graph:

1. A^1 gives the number of walks of length 1 (direct edges).
2. A^2 gives the number of walks of length 2.
3. A^3 gives the number of walks of length 3.
4. In general, the entry (i,j) of A^n gives the number of distinct walks of length n from vertex i to vertex j.

16 Implementing Shortest Path Algorithm for Unweighted Graphs

In an unweighted graph, all edges have the same cost (typically 1). Therefore, the Breadth-First Search (BFS) algorithm is used to find the shortest path because it explores vertices level by level, ensuring the minimum number of edges is traversed.

16.1 Breadth-First Search (BFS)

- Start from the source vertex and mark it as visited.
- Insert the source vertex into a queue.
- While the queue is not empty:
  1. Remove the front vertex from the queue.
  2. Visit all its unvisited adjacent vertices.
  3. Mark them as visited.
  4. Set their distance as the parent's distance + 1.
  5. Insert them into the queue.
- Continue until all reachable vertices are visited.

16.1.1 pseudo code

```
    BFS(Graph, Source)

1. Mark all vertices as unvisited.
2. Create an empty queue Q.
3. Mark Source as visited.
4. Enqueue(Source) into Q.

5. While Q is not empty do
      a. Vertex ← Dequeue(Q)
      b. Visit(Vertex)
```

```
    c. For each adjacent vertex U of Vertex do
          If U is not visited then
              Mark U as visited
              Enqueue(U)
          End If
       End For
End While
```

17 Weighted Graph

A weighted graph is a graph in which each edge is assigned a numerical value called a weight. The weight may represent distance, cost, time, capacity, or any other measurable quantity associated with traveling between two vertices.

17.1 Dijkatra's algorithm

Dijkstra's Algorithm is a greedy algorithm used to find the shortest path from a single source vertex to all other vertices in a weighted graph with non-negative edge weights.

- Assign a distance value of 0 to the source vertex and infinity to all other vertices.
- Mark all vertices as unvisited.
- Select the unvisited vertex with the smallest tentative distance.
- Mark the selected vertex as visited.
- For each adjacent vertex:
  1. Calculate the new distance:
     New Distance=Current Distance+Edge Weight
  2. If the new distance is smaller than the existing distance, update it.
- Repeat Steps 3–5 until all vertices are visited or no reachable vertices remain.

17.1.1 Pseudo code

```
    Initialize distance[source] = 0
Initialize distance[all other vertices] = infinty

While there are unvisited vertices:
    Select the vertex with the smallest distance
    Mark it as visited

    For each adjacent vertex:
        If current distance + edge weight < stored distance:
            Update the distance
```

18 Trees

A tree is a special type of graph that connects nodes in a hierarchical structure. It is one of the most important data structures in computer science and graph theory.
A tree is a connected, undirected graph with no cycles.
If a tree has n vertices, then it always has n-1 edges.

18.1 Properties of Trees

1. Connected: Every pair of vertices is connected by exactly one path.
2. Acyclic: A tree contains no cycles.
3. Unique Path: There is exactly one simple path between any two vertices.
4. Adding an extra edge creates exactly one cycle.

18.2 Terminology

1. Root: The topmost node in a rooted tree.
2. Parent: A node directly above another node.
3. Child: A node directly connected below a parent.
4. Leaf Node: A node with no children.
5. Internal Node: A node having at least one child.
6. Subtree: A tree formed by a node and all of its descendants.
7. Depth: Number of edges from the root to a node.
8. Height: Maximum depth of the tree.

18.3 Types of trees

1. Rooted Tree: One node is designated as the root.
2. Binary Tree: Each node has at most two children.
3. Binary Search Tree (BST): Left subtree contains smaller values, right subtree contains larger values.
4. Balanced Tree: Height is minimized to improve search efficiency (e.g., AVL Tree, Red-Black Tree).
5. Spanning Tree: A tree that connects all vertices of a connected graph without cycles.

19 Spanning trees

A spanning tree is a subgraph of a connected graph that includes all the vertices of the original graph and is itself a tree (connected and contains no cycles).
A spanning tree of a connected graph G(V,E) is a tree that:

1. Contains all the vertices of G.
2. Has exactly V-1 edges, where V is the number of vertices.
3. Contains no cycles and is connected.

A graph can have multiple edges.

20 Properties

1. Exists only if the graph is connected.
2. Contains all vertices of the original graph.
3. Has exactly $n - 1$ edges for n vertices.
4. Has no cycles.
5. There is exactly one path between every pair of vertices.
6. Removing any edge disconnects the spanning tree.
7. Adding any extra edge creates a cycle.

21 Minimum spanning trees

A Minimum Spanning Tree (MST) is a spanning tree of a connected, weighted graph that connects all the vertices with the minimum possible total edge weight.
A Minimum Spanning Tree (MST) is a spanning tree that:

1. Includes all vertices of the graph.
2. Has exactly $n - 1$ edges, where n is the number of vertices.
3. Contains no cycles.
4. Has the smallest possible sum of edge weights among all spanning trees.

21.1 Properties

1. Exists only for connected, weighted graphs.
2. Contains all the vertices.
3. Has exactly n-1 edges.
4. Contains no cycles.
5. The total weight is minimum among all spanning trees.
6. A graph can have more than one MST if different spanning trees have the same minimum total weight.

21.2 Disjoint set union

A Disjoint Set Union (DSU), also called the Union-Find data structure, is used to manage a collection of disjoint (non-overlapping) sets. It efficiently supports merging sets and checking whether two elements belong to the same set.
It is widely used in graph algorithms such as Kruskal's Minimum Spanning Tree (MST), network connectivity, and cycle detection.

21.2.1 Operations in DSU

- MakeSet(x)
  Creates a new set containing only the element x.
- Find(x)
  Returns the representative (parent/root) of the set containing element x.
  If two elements have the same representative, they belong to the same set.
- Union(x,y)
  Merges the sets containing x and y.

21.2.2 DSU representation

Usually represented using a parent array.
Element:1 2 3 4 5
Parent :1 1 3 3 5
Meaning:

1. 1 is root of 1,2
2. 3 is root of 3,4
3. 5 is alone

21.2.3 Optimisations

1. Path compression
   Whenever find() is called, every node on the path points directly to the root.
   Before
   4 — 3 — 2 — 1
   After
   4 — 1 2 3
   This greatly speeds up future operations.

2. Union by Rank (or Size)
   Instead of attaching trees randomly:

   - Attach the smaller tree under the larger tree.
   - Keeps the tree shallow.

21.2.4 Algorithms

1. Makeset

```
    for each vertex v
parent[v] = v
rank[v] = 0
```

2. Find

```
    Find(x):
if parent[x] != x
    parent[x] = Find(parent[x])
return parent[x]
```

3. Union

```
    Union(x, y):
rootX = Find(x)
rootY = Find(y)

if rootX == rootY
    return

if rank[rootX] < rank[rootY]
    parent[rootX] = rootY
else if rank[rootX] > rank[rootY]
    parent[rootY] = rootX
else
    parent[rootY] = rootX
    rank[rootX]++
```

21.2.5 Applications of DSU

1. Kruskal's Algorithm – Constructing the Minimum Spanning Tree (MST).
2. Cycle Detection – Detecting cycles in an undirected graph.
3. Dynamic Connectivity – Checking whether two nodes are connected.
4. Network Connectivity – Managing connected components in computer networks.
5. Image Processing – Connected component labeling.
6. Social Networks – Grouping users into connected communities.
7. Maze Generation – Ensuring all cells are connected without cycles.

21.3 Kruskal's algorithm

Kruskal's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph. It works by repeatedly selecting the smallest-weight edge that does not create a cycle until all vertices are connected.

22 Applications of Spanning Trees in Optimization and Networks

A spanning tree is a subset of a connected graph that includes all the vertices with the minimum number of edges (V-1) and contains no cycles. Spanning trees are widely used in optimization and communication networks because they reduce cost while maintaining connectivity.

- Network design
    1. Used to design computer networks, telephone networks, and power distribution systems.
    2. Ensures that all nodes are connected with the minimum number of cables or links.
    3. Reduces installation and maintenance costs.
- Minimum Cost Network (Minimum Spanning Tree)
    1. Algorithms like Kruskal's and Prim's find a Minimum Spanning Tree (MST).
    2. The MST connects all vertices with the minimum total edge weight.

    Applications:

    1. Road construction
    2. Railway networks
    3. Water pipeline systems
    4. Electrical power grids

- Communication Networks
    1. Used in LANs (Local Area Networks) to prevent loops.
    2. The Spanning Tree Protocol (STP) disables redundant paths while keeping backup links available.
    3. Prevents broadcast storms and ensures reliable data transmission.
- Routing and Broadcasting
    1. Efficiently broadcasts messages from one node to all other nodes.
    2. Since there are no cycles, duplicate message transmission is avoided.

  Example:Multicast communication in computer networks.

- Transportation Planning
    1. Helps design road, railway, and airline networks with minimum construction cost.
    2. Ensures every location is reachable while avoiding unnecessary routes.
- Power Distribution Systems
    1. Used in designing electrical grids.
    2. Connects substations with minimum transmission line cost while supplying power to all regions.
- Cluster Analysis and Data Mining
    1. MSTs are used in machine learning and data mining to group similar data points.
    2. Helpful in image segmentation and pattern recognition.
- Circuit Design
    1. Used in designing electronic circuits and printed circuit boards (PCBs).
    2. Minimizes wiring complexity and manufacturing cost.
- Network Reliability
    1. Helps analyze network connectivity.
    2. Used to determine critical connections whose failure may disconnect the network.
- Approximation Algorithms
    1. Spanning trees are used in approximation algorithms for difficult optimization problems such as:
        (a) Traveling Salesman Problem (TSP)
        (b) Steiner Tree Problem
        (c) Vehicle Routing Problems